INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Tecnología de Objetos				
TÍTULO DE LA PRÁCTICA:	C++ estructuras, clases y objetos				
NÚMERO DE PRÁCTICA:	01	**AÑO LECTIVO:**	2026-B	**NRO. SEMESTRE:**	VI (sexto)
FECHA DE PRESENTACIÓN	18/09/2026	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): Franco Jesus Cahua Soto				**NOTA:**	
DOCENTE(s): Mg. Ing. Rene Alonso Nieto Valencia					
Modalidad: Individual		Inicio de la práctica: 11/09/2026			

SOLUCIÓN Y RESULTADOS

I. INTRODUCCIÓN

El paradigma de Programación Orientada a Objetos (POO) y el lenguaje C++ constituyen una de las herramientas más robustas y deterministas para el diseño e implementación de sistemas de software de alto rendimiento. Frente a la programación estructurada tradicional, la orientación a objetos permite modelar entidades del mundo real mediante la abstracción, encapsulando estado (atributos) y comportamiento (métodos u operaciones) en unidades coherentes y reutilizables.

En la presente práctica de laboratorio **N.° 01** de la asignatura **Tecnología de Objetos**, se abordan los fundamentos esenciales de C++, iniciando desde la sintaxis de flujos estándar de entrada/salida (`std::cin` y `std::cout`), el formateo riguroso de reportes en consola mediante la biblioteca de manipulación `<iomanip>`, y la transición fundamental entre las estructuras de datos heterogéneas (`struct`) y las clases formales (`class`) con niveles de visibilidad y encapsulamiento.

SOLUCIÓN Y RESULTADOS

I.1 Objetivos de la práctica

- Dominar el uso de los operadores de flujo estándar de entrada y salida (`cin` y `cout`) en consola.
- Comprender y aplicar la generación de números pseudoaleatorios empleando `srand()` y la función de reloj del sistema.
- Formatear salidas tabulares complejas empleando el manipulador `setw()` y evaluar críticamente alternativas de formateo.
- Modelar gráficamente entidades complejas mediante diagramas de clases UML con tres compartimentos: nombre, atributos y operaciones.
- Implementar modelos de entidades de software mediante `struct` con múltiples métodos asociados y contrastarlos frente a implementaciones completas en `class` bajo encapsulamiento formal.

II. MARCO CONCEPTUAL

II.1 Sentencias de programa y flujo de compilación en C++

La sentencia es la unidad elemental de ejecución en C++. Cada instrucción concluye obligatoriamente con punto y coma (`;`). El preprocesador analiza las directivas iniciadas con el carácter numeral (`#include`), las cuales incorporan antes de la etapa de traducción los archivos de cabecera (*header files*) que definen las firmas de funciones, plantillas y clases estándar.

II.2 Directiva `using` y Espacios de Nombres (`namespace`)

Para evitar colisiones de identificadores en proyectos a gran escala, C++ agrupa los componentes de su biblioteca estándar dentro del espacio de nombres `std`. La invocación de identificadores puede realizarse de dos formas:

1. **Cualificación explícita:** Prefijando el operador de resolución de ámbito (`::`), por ejemplo `std::cout` y `std::cin`, garantizando máxima claridad y aislamiento modular.
2. **Directiva de apertura:** Declarando `using namespace std;`, lo cual importa todos los símbolos al espacio de nombres actual, reduciendo la verbosidad en programas pequeños pero incrementando el riesgo potencial de ambigüedad.

II.3 Sobrecarga de Operadores de Flujo

En contraste con las funciones históricas de C como `printf()` y `scanf()`, C++ introduce flujos fuertemente tipados a través de las clases `std::istream` y `std::ostream`. El operador de desplazamiento binario a izquierda (`«`) se sobrecarga como operador de inserción en flujo, mientras que el operador a derecha (`»`) se sobrecarga como *operador de extracción*. La adaptación tipológica es automática y no requiere máscaras de conversión manuales.

SOLUCIÓN Y RESULTADOS

II.4 Manipuladores de Entrada y Salida (<iomanip>)

Para acondicionar la presentación visual de datos en pantalla, C++ dispone de la biblioteca <iomanip>, destacando:

- `std::setw(int n)`: Define el ancho del campo para la siguiente operación de inserción. Su efecto es no permanente (efímero) y se restablece tras procesar una sola variable.
- `std::left` y `std::right`: Modifican la justificación del texto dentro del ancho asignado.
- `std::fixed` y `std::setprecision(int p)`: Controlan el formato decimal en coma fija y el número de cifras fraccionarias.
- `std::setfill(char c)`: Sustituye el espacio en blanco por el carácter especificado para rellenar campos.

II.5 Diferencias arquitectónicas entre `struct` y `class`

En el lenguaje C++, tanto `struct` como `class` representan tipos definidos por el usuario capaces de almacenar variables miembro y funciones miembro (métodos). La única distinción técnica radica en la visibilidad por defecto:

- En una `struct`, todos los miembros y la herencia son **públicos** (`public`) por defecto.
- En una `class`, todos los miembros y la herencia son **privados** (`private`) por defecto, forzando la implementación de principios de encapsulamiento estricto.

III. ANÁLISIS DE EJERCICIOS RESUELTOS POR EL DOCENTE

En la sesión práctica se evaluaron diversos casos de estudio orientados a asimilar la sintaxis y comportamiento del lenguaje.

III.1 Directivas de compilación y espacios de nombres

Se analizó el programa elemental de emisión de texto contrastando el uso directo con la directiva `using namespace std;`:

Listado 1: Comparativa de emisión con y sin resolución de ámbito explícita

```cpp
#include <iostream>
using namespace std;

int main() {
    // Emision utilizando el namespace global importado
    cout << "Esto es una cadena de caracteres!\n";

    // Emision con cualificacion explicita segura
    std::cout << "Esto es una cadena de caracteres!\n";
    return 0;
}
```

SOLUCIÓN Y RESULTADOS

Conclusión analítica: La cualificación explícita `std::` representa la mejor práctica en desarrollo profesional de software al evitar la polución del espacio global de nombres.

III.2 Tamaños de tipos de variables en memoria (`sizeof`)

Se verificó el tamaño de almacenamiento en bytes de las variables fundamentales bajo arquitectura x86-64:

Tipo de Dato	Bytes (`sizeof`)	Rango de Representación
`char`	1 byte	-128 a 127 (ASCII extendido: 0 a 255)
`short int`	2 bytes	$-32{,}768$ a $32{,}767$
`int`	4 bytes	$-2{,}147{,}483{,}648$ a $2{,}147{,}483{,}647$
`long int` (x86_64)	4 u 8 bytes	Acorde al modelo de datos (LLP64 en Windows = 4B, LP64 en Linux = 8B)
`long long`	8 bytes	-9×10^{18} a 9×10^{18}
`float`	4 bytes	Precisión simple IEEE 754 (≈ 7 dígitos decimales)
`double`	8 bytes	Doble precisión IEEE 754 ($\approx 15 - 17$ dígitos decimales)

III.3 Manipulador `setw` con variables de población

Se evaluó el alineamiento tabular presentado por el docente para localidades y poblaciones:

Listado 2: Formateo de poblaciones usando setw docente

```
long pop1 = 2425785, pop2 = 47, pop3 = 9761;
cout << setw(12) << "UBICACION" << setw(12) << "POB." << endl
     << setw(12) << "Lima"      << setw(12) << pop1   << endl
     << setw(12) << "Atora"     << setw(12) << pop2   << endl
     << setw(12) << "Sicuani"   << setw(12) << pop3   << endl;
```

El manipulador `setw(12)` garantiza que cada cifra y etiqueta ocupe exactamente 12 columnas en el flujo, alineando automáticamente los números a la derecha y manteniendo la coherencia vertical.

III.4 Operador Remanente (`%`)

Se analizó la operación de residuo aritmético en números enteros:

$$9 \pmod 8 = 1 \quad \text{y} \quad 10 \pmod 8 = 2 \tag{1}$$

Dicho operador resulta vital en algoritmos de dispersión (*hashing*), generación de números aleatorios en un rango $[1, N]$ acotado mediante `(rand() % N) + 1`, y control de índices cíclicos en estructuras de datos lineales.

IV. DESARROLLO DE EJERCICIOS PROPUESTOS

A continuación se detalla la solución completa a los 5 ejercicios propuestos en la guía de laboratorio, acompañada de los fundamentos técnicos, códigos C++ verificados y capturas de ejecución.

SOLUCIÓN Y RESULTADOS

IV.1 Ejercicio 1: Generación y Ordenamiento Descendente en C++

Enunciado: Desarrollar un programa en C++ que utilice flujos `cin` y `cout`. Debe solicitar al usuario un número entero positivo N. A continuación solicita un carácter: si es `'g'`, autogenera N números aleatorios entre 1 y N empleando `srand()`; si es `'i'`, permite ingresar manualmente N números. Finalmente, ordena los elementos de mayor a menor empleando un algoritmo eficiente de ordenamiento.

Diseño de la solución: Se seleccionó el algoritmo **Quicksort descendente**. Para garantizar una distribución pseudoaleatoria real en cada ejecución en modo `'g'`, se inicializa la semilla mediante `std::srand(std::time(nullptr))`. La fórmula de traslación empleada es `(std::rand() % N) + 1`.

Listado 3: Función de partición descendente para Quicksort en ejercicio1.cpp

```cpp
int particionDescendente(std::vector<int>& arr, int bajo, int alto) {
    int pivote = arr[alto];
    int i = bajo - 1;
    for (int j = bajo; j < alto; ++j) {
        // Criterio de orden descendente: elementos mayores a la izquierda
        if (arr[j] >= pivote) {
            ++i;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[alto]);
    return i + 1;
}
```

Resultado de la ejecución experimental: Se ingresó $N = 8$ en modalidad de autogeneración (`'g'`):

Estado	Valores del Arreglo
Entrada generada ($N = 8$, intervalo $[1, 8]$)	`[ 3, 6, 3, 5, 5, 2, 7, 6 ]`
Salida ordenada (Mayor a Menor)	`[ 7, 6, 6, 5, 5, 3, 3, 2 ]`

IV.2 Ejercicio 2: Estadísticas Deportivas con `setw` y Alternativas

Enunciado: Crear un programa en C++ que utilice `setw` para listar una estadística deportiva propuesta de forma ordenada. Luego, reutilizar el código sustituyendo `setw` por otro mecanismo o manipulador investigado. Responder analíticamente: ¿Cuál ha servido mejor?

Diseño de la solución: Se modeló la **Tabla de Posiciones de la Liga 1 de Fútbol Profesional**, que calcula Partidos Jugados (PJ), Ganados (PG), Empatados (PE), Perdidos (PP), Goles a Favor (GF), Goles en Contra (GC), Diferencia de Goles (DG), Puntos (PTS) y Porcentaje de Eficacia.

Listado 4: Comparativa de formateo en ejercicio2.cpp

```cpp
// Version 1: Formateo estricto con std::setw
std::cout << std::left  << std::setw(5)  << eq.posicion
          << std::left  << std::setw(22) << eq.equipo
          << std::right << std::setw(5)  << eq.pj
```

SOLUCIÓN Y RESULTADOS

```
            << std::right << std::setw(6)  << eq.pts
            << std::right << std::setw(9)  << std::fixed
            << std::setprecision(1) << eq.eficacia << "%\n";

// Version 2: Formateo reutilizado alternativo con especificadores de mascara
std::printf("%-5d%-22s%5d%6d%9.1f%%\n",
             eq.posicion, eq.equipo.c_str(), eq.pj, eq.pts, eq.eficacia);
```

Resultado tabular en consola:

POS	EQUIPO	PJ	PG	PE	PP	GF	GC	DG	PTS	EFIC (%)
1	Universitario	17	12	4	1	32	10	+22	40	78.4 %
2	Alianza Lima	17	11	4	2	28	11	+17	37	72.5 %
3	Sporting Cristal	17	10	4	3	35	15	+20	34	66.7 %
4	FBC Melgar	17	9	5	3	30	16	+14	32	62.7 %
5	Cusco FC	17	9	4	4	26	20	+6	31	60.8 %

Evaluación técnica: ¿Cuál ha servido mejor?

- **Análisis de** `std::setw`**:** Ofrece seguridad estricta de tipos (*type safety*) y se integra orgánicamente con sobrecargas de clases C++. No obstante, su comportamiento no persistente obliga a invocar `std::setw(n)` en cada columna individual, produciendo código extenso y repetitivo.
- **Análisis del formateo alternativo (**`printf` **/ cadenas con máscara):** Permite condensar la alineación, el ancho mínimo y la precisión decimal en una única cadena compacta (ej. `%-5d%-22s%5d%9.1f%%`).
- **Veredicto:** Para tablas complejas de múltiples columnas, el formateador con máscaras resulta notablemente más legible y mantenible, mientras que `std::setw` es indispensable cuando se requiere aislamiento y seguridad estricta de tipos en flujos orientados a objetos.

IV.3 Ejercicio 3: Propuesta Gráfica de Clase para Citas Médicas

Enunciado: Proponer en forma gráfica una clase con atributos, referido a citas médicas de un hospital (ejm. Cita fecha/hora, tipo atención, número del asegurado, médico, costo, etc.).

Modelo gráfico propuesto: La entidad se representa mediante la convención formal de tres compartimentos rectangulares:

SOLUCIÓN Y RESULTADOS

CitaMedicaHospital

Atributos:
- idCita: int
- numeroAsegurado: string
- nombrePaciente: string
- especialidad: string
- medicoAsignado: string
- fechaHora: string
- tipoAtencion: string *(Consulta Externa, Urgencia, Cirugía)*
- tipoSeguro: TipoSeguro *(Particular, EsSalud, SIS, EPS)*
- costoBase: double
- costoFinal: double
- estadoCita: string *(Registrada, Confirmada, Atendida, Cancelada)*

Operaciones / Métodos:
+ inicializar/constructor(...): void
+ calcularCostoFinal(): double
+ reprogramarCita(nuevaFecha, nuevaHora): bool
+ cancelarCita(motivo): bool
+ esAtencionUrgente(): bool
+ imprimirBoletaAtencion(): void

IV.4 Ejercicio 4: Implementación de Cita Médica con Estructura (`struct`)

Enunciado: Implementar en C++ el modelo de citas médicas anterior utilizando una estructura (`struct`) con más de 02 métodos.

Diseño e implementación: Se desarrolló en `ejercicio4.cpp` la estructura `CitaMedica` con 5 métodos: `inicializar()`, `calcularCostoNeto()`, `mostrarTicketAtencion()`, `reprogramarCita()` y `confirmarAtencion()`.

Listado 5: Estructura CitaMedica con métodos en ejercicio4.cpp

```
struct CitaMedica {
    int idCita;
    std::string numeroAsegurado, nombrePaciente, especialidad;
    std::string medicoAsignado, fecha, hora, tipoAtencion, estado;
    double costoBase, costoFinal;

    void inicializar(int id, std::string nAseg, std::string pac,
                     std::string esp, std::string med, std::string f,
                     std::string h, std::string tipo, double costo) {
        idCita = id; numeroAsegurado = nAseg; nombrePaciente = pac;
        especialidad = esp; medicoAsignado = med; fecha = f; hora = h;
        tipoAtencion = tipo; costoBase = costo; costoFinal = costo;
        estado = "PENDIENTE";
    }

    double calcularCostoNeto(double cobertura) {
        costoFinal = costoBase * (1.0 - cobertura);
```

SOLUCIÓN Y RESULTADOS

```
        return costoFinal;
    }

    void reprogramarCita(const std::string& nF, const std::string& nH) {
        fecha = nF; hora = nH; estado = "REPROGRAMADA";
    }
    // Metodos mostrarTicketAtencion() y confirmarAtencion()...
};
```

Demostración de ejecución de ticket generado por la estructura:

```
+--------------------------------------------------------------+
|              HOSPITAL NACIONAL - TICKET DE CITA              |
+--------------------------------------------------------------+
| ID Cita:            #101                                     |
| Nro. Asegurado:     ASEG-849201                              |
| Paciente:           Franco Cahua                             |
| Especialidad:       Cardiologia                              |
| Medico:             Dr. Roberto Valdivia                     |
| Fecha y Hora:       15/10/2026 - 09:30                       |
| Tipo de Atencion:   Consulta Externa                         |
| Estado de Cita:     CONFIRMADA                               |
+--------------------------------------------------------------+
| Costo Base:         S/.   120.00                             |
| Costo a Pagar:      S/.    24.00 (Cobertura Seguro 80%)      |
+--------------------------------------------------------------+
```

IV.5 Ejercicio 5: Implementación de Cita Médica con Clases y POO

Enunciado: Implementar en C++ el modelo de citas médicas utilizando clases (`class`), aplicando encapsulamiento y métodos formales.

Diseño e implementación: Se desarrolló la clase `CitaMedicaHospital` en `ejercicio5.cpp`. Los miembros de datos son estrictamente `private`, protegiendo el estado de la entidad frente a modificaciones arbitrarias externas. El acceso y manipulación se gestiona mediante constructores parametrizados, métodos observadores (*getters*) y métodos de negocio.

Listado 6: Clase con encapsulamiento en ejercicio5.cpp

```
class CitaMedicaHospital {
private:
    int idCita;
    std::string numeroAsegurado, paciente, especialidad, medicoAsignado;
    std::string fecha, hora, tipoAtencion, estado, motivoCancelacion;
    TipoSeguro tipoSeguro;
    double costoBase, costoFinal;

public:
    CitaMedicaHospital(int id, const std::string& nAseg, const std::string& pac,
                       const std::string& esp, const std::string& med,
                       const std::string& f, const std::string& h,
```

SOLUCIÓN Y RESULTADOS

```
                        const std::string& tipo, TipoSeguro seguro, double costo)
        : idCita(id), numeroAsegurado(nAseg), paciente(pac), especialidad(esp),
          medicoAsignado(med), fecha(f), hora(h), tipoAtencion(tipo),
          tipoSeguro(seguro), costoBase(costo), estado("REGISTRADA") {
        calcularCostoFinal();
    }

    double calcularCostoFinal() {
        double cobertura = (tipoSeguro == TipoSeguro::SIS) ? 1.00 :
                           (tipoSeguro == TipoSeguro::PRIVADO_EPS) ? 0.85 :
                           (tipoSeguro == TipoSeguro::ESSALUD) ? 0.70 : 0.00;
        costoFinal = costoBase * (1.0 - cobertura);
        return costoFinal;
    }
};
```

Salida de boleta hospitalaria generada en la prueba:

```
================================================================
                SISTEMA DE GESTION HOSPITALARIA
              BOLETA DE ATENCION DE CITA MEDICA
================================================================
  Numero de Cita:       #201
  Codigo Asegurado:     00481239-01
  Paciente:             Franco Jesus Cahua Soto
  Tipo de Seguro:       EPS Privado (85% cobertura)
  Especialidad:         Neurologia
  Medico Tratante:      Dr. Victor Zegarra
  Fecha programada:     19/10/2026   |   Hora: 16:30
  Tipo de Servicio:     Consulta Externa [REGULAR]
  Estado Actual:        REPROGRAMADA
----------------------------------------------------------------
  Costo Arancelario:    S/. 150.00
  Descuento Cobertura:  S/. 127.50
  TOTAL A COBRAR:       S/. 22.50
================================================================
```

V. CUESTIONARIO

V.1 Descripción de las bibliotecas comúnmente utilizadas en C y C++

1. `<iostream>` **(Input/Output Stream en C++):** Define los objetos fundamentales de flujo estándar en C++: `std::cin` (entrada estándar, descriptor 0), `std::cout` (salida estándar, descriptor 1 con búfer), `std::cerr` (salida de error sin búfer) y `std::clog` (registro de eventos con búfer). Proporciona tipado estricto en tiempo de compilación y extensibilidad a través de la sobrecarga de los operadores `«` y `»`.

2. `<cstdlib>` **/** `<stdlib.h>` **(Standard General Utilities Library):** Biblioteca heredada de C que contiene utilidades fundamentales del sistema operativo:

SOLUCIÓN Y RESULTADOS

- **Gestión de memoria dinámica:** Funciones clásicas `malloc()`, `calloc()`, `realloc()` y `free()` (sustituidas en POO moderna por los operadores `new` y `delete`).
- **Control de ejecución y procesos:** `exit()`, `abort()`, `system()`.
- **Generación pseudoaleatoria:** `rand()` (generador congruencial lineal) y `srand()` (establecimiento de la semilla inicial).
- **Conversión numérica:** `atoi()`, `atof()`, `strtol()`, `strtod()`.

3. `<cstdio>` **/** `<stdio.h>` **(C Standard Input and Output):** Implementa el modelo clásico de operaciones de entrada y salida basado en flujos de punteros a archivo (`FILE*`). Incluye `printf()`, `scanf()`, `sprintf()`, `snprintf()`, `fopen()`, `fclose()`, `fread()` y `fwrite()`. Aunque carece de verificación de tipos en tiempo de compilación, destaca por su elevada velocidad de transferencia en bloques contiguos de bytes.
4. `<iomanip>` **(Input/Output Parametric Manipulators):** Proporciona funciones y objetos diseñados para parametrizar el comportamiento interno de los flujos de `<iostream>`, permitiendo especificar la anchura de impresión (`setw`), la precisión en coma flotante (`setprecision`), el relleno de caracteres (`setfill`), y la base numérica de visualización (`hex`, `dec`, `oct`).

VI. CONCLUSIONES

1. La arquitectura de flujos de C++ (`std::cin` y `std::cout`) proporciona un esquema de entrada y salida fuertemente tipado que reduce sustancialmente los errores de ejecución frente a funciones de conversión implícita heredadas de C.
2. La biblioteca `<iomanip>` y en particular el manipulador `setw()` permiten estructurar interfaces por consola rigurosamente alineadas; no obstante, su naturaleza no persistente aconseja evaluar alternativas complementarias para tablas de alta densidad.
3. Aunque en C++ tanto `struct` como `class` admiten funciones miembro y constructores, la convención de ingeniería y el principio de encapsulamiento dictan que `class` debe reservarse para entidades con estado protegido y lógica de negocio sensible, garantizando la invariancia de los datos.
4. El modelado gráfico de entidades mediante diagramas de clases UML de tres compartimentos establece un puente claro y directo entre la fase de diseño de software y su codificación determinista en C++.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Universidad Nacional de San Agustín. (2026). *Guía de Práctica de Laboratorio 01: C++ estructuras, clases y objetos*. Escuela Profesional de Ingeniería de Sistemas. Código: GUIA-PRLD-001. Docente: Mg. Ing. Rene Alonso Nieto Valencia.

[2] Stroustrup, B. (2013). *The C++ Programming Language* (4.ª ed.). Addison-Wesley Professional.

[3] Deitel, P., y Deitel, H. (2017). *C++ How to Program* (10.ª ed.). Pearson.

[4] Laaksonen, A. (2017). *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer.

[5] Meyers, S. (2005). *Effective C++: 55 Specific Ways to Improve Your Programs and Designs* (3.ª ed.). Addison-Wesley.